

Credit Risk Desk: ML Credit Card Approval Screening

Project Description

Credit Risk Desk screens credit card applications for future delinquency risk using a hybrid system: a deterministic rule-based eligibility gate for inputs the model has no basis to judge (e.g. zero income), followed by a tuned XGBoost model for risk-ranking applicants who pass the gate, followed by SHAP-based reason codes so every score is explainable. The platform includes an application intake form, an automated screening engine, and a result view with a risk gauge.

This replaces an earlier version that called the ML model directly on raw form input at a default 0.5 threshold — which meant a Rs. 0-income applicant could be scored as low-risk purely because the training data contains no such example to learn from.

Scenarios

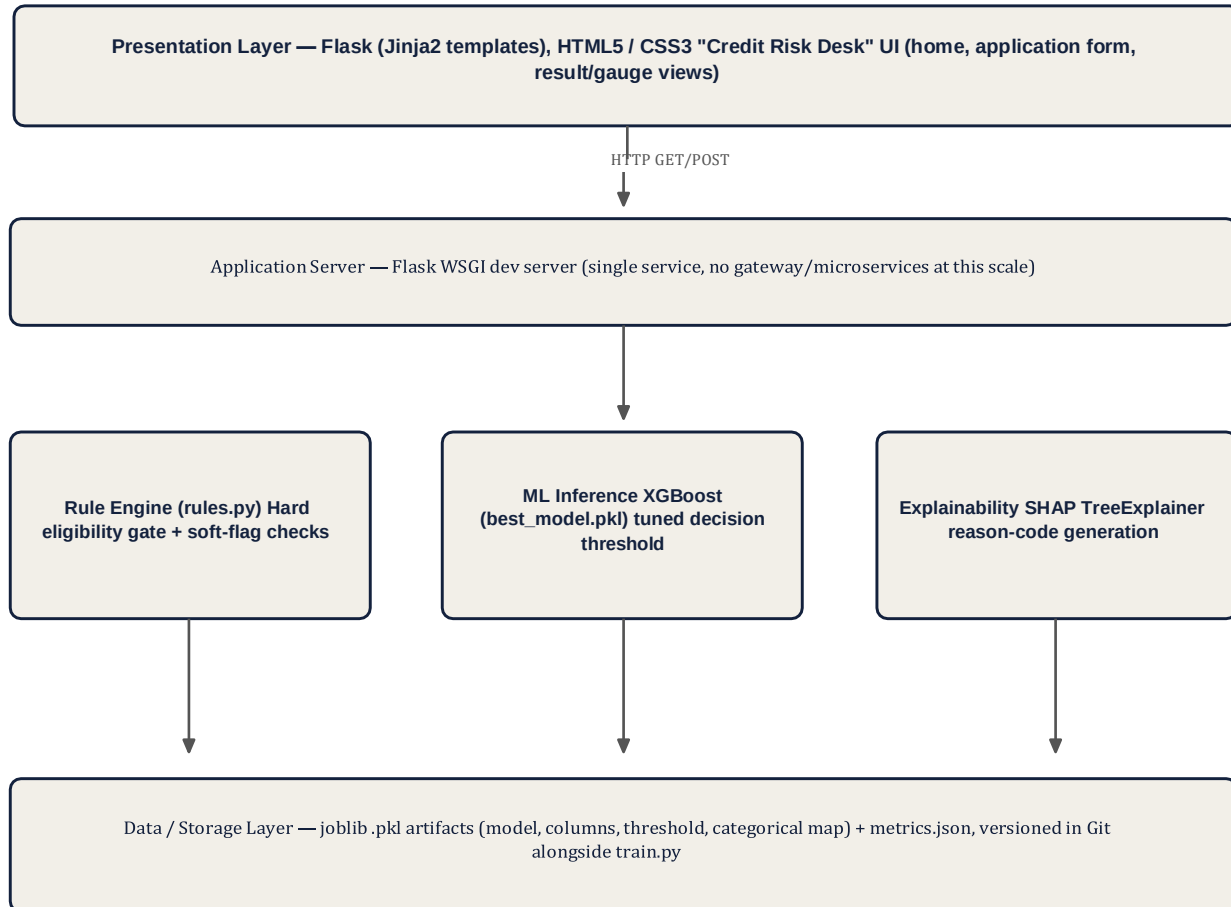
Scenario 1: An applicant enters Rs. 0 annual income. The rule gate declines the application immediately with the reason "No verifiable income reported" — the ML model is never called, since it has no training examples anywhere near this input.

Scenario 2: A well-qualified applicant (stable income, property ownership, long employment tenure) passes the gate and receives a low risk score; the result page shows "Annual income lowers estimated risk" and "Income per family member lowers estimated risk" as the top SHAP-driven reasons.

Scenario 3: A borderline applicant (large family, short employment tenure, no property) clears the gate but crosses the tuned decision threshold, and is flagged for manual review with reason codes pointing to family size and employment stability.

Technical Architecture

Description: A single Flask service renders the UI and orchestrates two stages — a rule engine (rules.py) that can short-circuit to a decline, and an ML inference + SHAP explanation stage (app.py) for applicants who pass the gate. Model artifacts (best_model.pkl, model_columns.pkl, decision_threshold.pkl, categorical_value_map.pkl) are versioned flat files produced by train.py, not a database — there is no persistent applicant data store in the current version.



Project Workflow

Milestone 1: Data & Baseline Modelling

- Activity 1.1: Merge application_record.csv and credit_record.csv on ID; define the high-risk target via vintage analysis (60+ days overdue).
- Activity 1.2: Engineer AGE_YEARS, YEARS_EMPLOYED, INCOME_PER_PERSON, LOG_INCOME, EMPLOYMENT_STABILITY.
- Activity 1.3: Train and compare Logistic Regression, Decision Tree, Random Forest, and XGBoost baselines.

Milestone 2: Rule Gate, Threshold Tuning & Explainability

- Activity 2.1: Design and implement the hard-eligibility rule gate (rules.py).
- Activity 2.2: Tune the XGBoost model with RandomizedSearchCV under 5-fold stratified CV.
- Activity 2.3: Select a decision threshold via F2-optimization on a validation split.
- Activity 2.4: Integrate SHAP TreeExplainer and filter reason codes to the applicant's actual attributes.

Milestone 3: UI, Testing & Documentation

- Activity 3.1: Redesign the Flask UI ("Credit Risk Desk" theme) with a risk gauge and ledger-style stats.
- Activity 3.2: Run in-process latency and concurrency testing via the Flask test client.

Activity 3.3: Document requirements, architecture, planning, testing and demo materials